

Hanna Little-Neuhaus

Nicholas Krustchinsky

MIPS Processor Project

NATHANIEL JONES

Victor Coleman

What it is

- A cycle-accurate simulator of a 5-stage pipelined MIPS processor written in C++
- You give it a MIPS assembly file as input, and it runs it through the simulated pipeline and outputs the final state of all 32 registers and memory when the program finishes.
- It has two modes. Normal mode just gives you the final output. Debug mode prints the contents of every pipeline register and the register file after every single clock cycle, so you can watch the program execute instruction by instruction.

Key Components

Assembler

2-pass: builds label table, then encodes to binary

Encoder

Converts each instruction to 32-bit binary (R / I / J format)

ControlUnit

Generates control signals for each instruction type

ALU

Executes all 14 supported MIPS instructions

RegisterFile

32 registers, \$zero hardwired to 0

Memory

Word-addressable, dynamic map — only stores written locations

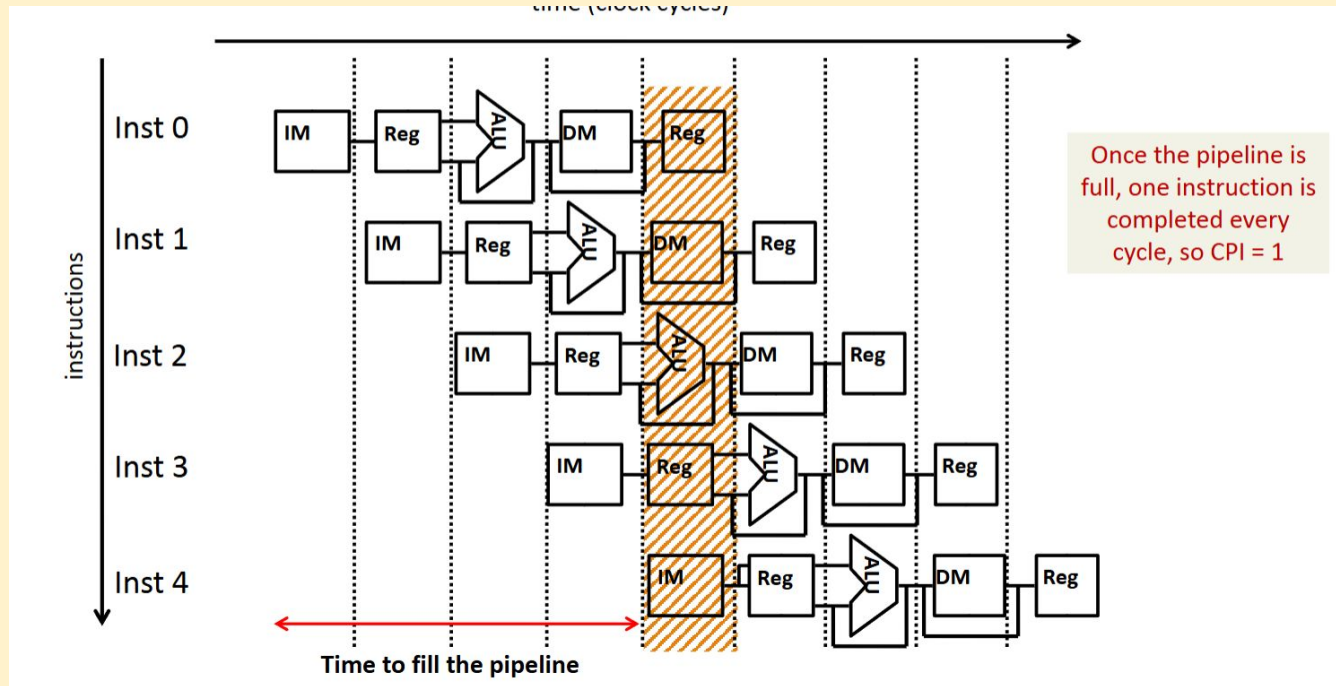
PipelineRegisters

4 structs: IF/ID, ID/EX, EX/MEM, MEM/WB

Simulator

Clock loop — drives all 5 stages each cycle

Datapath



Assembler: Why?

Assembly code “add \$t0, \$t1, \$t2”
does not belong on the CPU.

So, need to convert assembly
code into machine code *before*
even considering the datapath!



Assembler: How?

Must assemble / “encode”
assembly to machine code.

So, iterate the input file, convert
stuff, easy right?

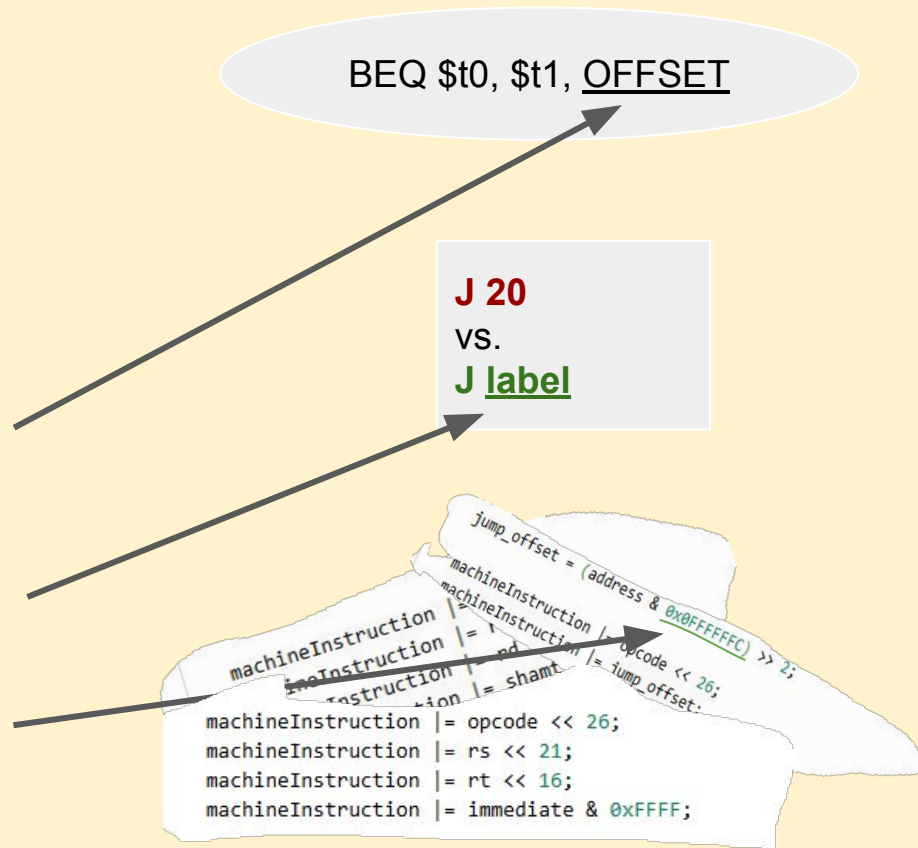
Well...



Encoding

Not so easy! Need a few things:

- “PC” at instruction: needed to encode PC-relative address
- “PC” at label: for jump/branch targets; need a “label table”
- Quite a bit of bit-manipulation
- Way too many conditionals.



Instruction Fetch

Get the instruction (if it exists)

- PC counting in bytes
- program vector<uint32_t> stores instruction as words?

Words are just 4 bytes in a trenchcoat.
>> shift right 2

```
void Simulator::stageIF() {  
    int wordPC = pc >> 2;  
    if (wordPC < (int)program.size()) {  
        ifid.instruction = program[wordPC];  
        ifid.valid = true;  
        ifid.pc = pc;  
        pc += 4;  
    } else {  
        ifid.instruction = 0;  
        ifid.valid = false;  
    }  
}
```


What the is that? Instruction Decode

Instruction encode is hard

Converting english to int sucks, but it pays off...

instructions.bin

1

DC1 BS NULSOH ! (P NULNUL @ BS NULNULACK ! BS NULENQSOH * @ CAN

Instruction decode is easy

...instructions following specification decode FAST!

// All values will be populated... not all will be used.

idx.opcode = (ifid.instruction >> 26) & 0x3F;

idx.rs = (ifid.instruction >> 21) & 0x1F;

idx.rt = (ifid.instruction >> 16) & 0x1F;

idx.rd = (ifid.instruction >> 11) & 0x1F;

idx.shamt = (ifid.instruction >> 06) & 0x1F;

idx.funct = (ifid.instruction >> 00) & 0x3F;

int16_t intermediate_immediate = ifid.instruction & 0x0000FFFF;

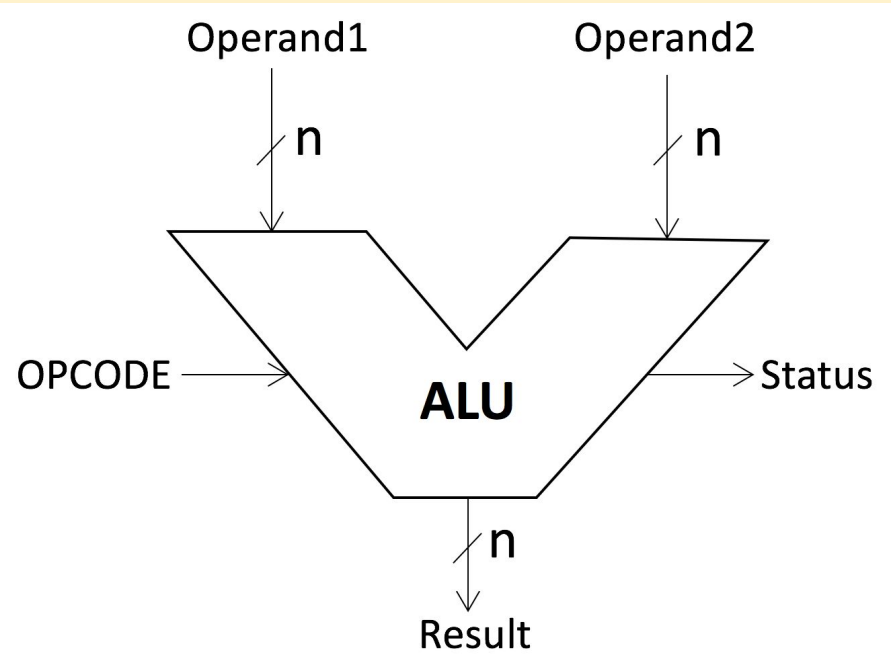
idx.immediate = (int32_t)intermediate_immediate; // auto extend?

idx.jump_offset = ifid.instruction & 0x03FFFFFF;

Instruction Execution

This stage of the pipeline handles the actual operations and makes use of the ALU, which has the math and logic operations.

It passes forward the control signals, and handles BEQ so that some things don't have to be passed along further.



The ALU

The ALU uses enum for the opcodes in order to create a switch statement.

```
struct ALUResult {  
    int value; //output of the function  
    bool zero; //zero flag. if output = 0, this is true.  
};  
  
class ALU {  
public:  
    //op : the operation  
    //a : first input  
    //b : second input  
    ALUResult execute(Opcode op, int a, int b);  
};
```

```
enum class Opcode {  
    ADD, ADDI, SUB, MUL,  
    AND, OR,  
    SLL, SRL,  
    LW, SW, BEQ, J, NOP  
};
```

The ALU returns all of its information in a struct which contains the output and a flag for 0.

The actual implementation uses one of the enum ops and two input vals to get the value.

RegisterFile

All registers are set at 0 for reset

While read() takes 2 combination read ports inputs from the either \$rs or \$rt in the ID stage of the pipeline.

Prevents writing to zero

dump() is the debug feature that allows register state observation after the execution.

```
int32_t RegisterFile::read(int regNum) const {
    if (regNum < 0 || regNum >= 32){
        std::cerr << "RegisterFile:: read - Invalid register number: " << regNum <<
        return 0;
    }
    if(regNum == 0) return 0; // $Zero register stays zero
    return regs[regNum];
}

void RegisterFile::write(int regNum, int value) {
    if (regNum < 0 || regNum >= 32){
        std::cerr << "RegisterFile:: write - Invalid register number: " << regNum <
        return;
    }
}
```

Control Unit

Controls signal flow flow by:

Operating Decode stage and translation its instruction fields into control signal.

Which each instruction type in activated when when the the speigic combination is matched through conditional logic.

Signals then stored in ID/EX pipeline register and pushed to next stage EX, Mem, WB

```
//reset everything since compulsory miss upon
idex.regDst = false;
idex.aluSrc = false;
idex.memRead = false;
idex.memWrite = false;
idex.regWrite = false;
idex.memToReg = false;

uint32_t op = idex.opcode;
```

```
//LW
else if(op == opcodeMap.at("lw")){
    idex.aluSrc = true;
    idex.memRead = true;
    idex.regWrite = true;
    idex.memToReg = true;
    idex.ALUOp = Opcode::LW;
}
//BEQ
else if(op == opcodeMap.at("beq")){
    idex.branch = true;
    idex.ALUOp = Opcode::BEQ;
}
```

PipelineRegisterFile:

```
ID_EX() : valid(false), readData1(0), readData2(0), immediate(0), type(NOP),  
         rs(0), rt(0), rd(0), shamt(0), funct(0), opcode(0),  
         regDst(false), aluSrc(false), memRead(false), memWrite(false),  
         regWrite(false), memToReg(false), branch(false), ALUOp(opcode::NOP) {}
```

```
int opcode; // This is now a number  
  
// control signals  
bool regDst;      // 1 = write to  
bool aluSrc;      // 1 = use immed  
bool memRead;     // 1 = this inst  
bool memWrite;    // 1 = this inst  
bool regWrite;    // 1 = this inst  
bool memToReg;    // 1 = writeback  
bool branch;      // 1 = branch in  
Opcode ALUOp;     // Not an "Opcod
```

- Boolean control value signals to help determine the instruction type used later.
- Initializes all 5 stage values with 0.

DEMO

Expect to see some working (and not working) demonstrations.

Questions?